

Lab	Practical
LAB-1	Git – Local Repository - Git Installation and Repository Initialization. - Verify Git installation. - Configure username and email. - Create a folder GitLab. - Initialize it as a Git repository. - Check repository status. - Create a file README.md and add basic project information - Tracking Files and Creating Commits. - Create files: index.html, style.css, script.js - Stage all files and commit the changes. - View commit history. - Understanding File States and Status. - Modify index.html. - Create about.html. - Check repository status. - Stage only one file. - Compare status before and after staging. - Undoing Changes. - Modify style.css. - Restore original version. - Create and commit another change. - Undo the latest commit while keeping changes. - Undo the latest commit permanently.
LAB-2	Git Branch - Branch Creation and Management. - Create branch feature-login. - Switch to it. - Add a file login.html. - Commit the changes. - Display all branches. - Merging Branches. - Create branch feature-contact. - Add contact.html. and commit changes. - Switch to main branch. - Merge feature branch. - Verify merged content. - Merge Conflict Resolution. - Create branch feature-ui. - Modify same line in README.md. - Modify same line in main branch.

	d. Attempt merge. e. Resolve conflict manually. f. Complete merge.
LAB-3	GitHub Remote Repository 1. GitHub Remote Repository Integration. a. Create repository on GitHub. b. Connect local repository. c. Push code. d. Verify repository online. 2. GitHub Collaboration Workflow. a. Create branch feature-profile. b. Add profile page. c. Push branch to GitHub. d. Create Pull Request. e. Merge Pull Request. f. Pull latest changes to local repository. 3. Advanced Git – Rebase and Reflog a. Create branch feature-dashboard. b. Add two commits. c. Rebase branch onto main. d. View reflog history. e. Recover a lost commit using reflog.
LAB-4	Docker 1. Getting Started with Docker. a. Verify Docker installation. Display Docker version and system information. b. Pull the hello-world image. Run the container. c. List available images and containers. 2. Working with Containers. a. Run an Ubuntu container interactively. b. Create a file inside the container. c. Exit and restart the container. d. Remove the container. 3. Exploring Docker Images. a. Pull Ubuntu and Nginx images. b. Inspect image details. c. Remove unused images. d. Compare image sizes.
LAB-5	Docker 1. Networking and Port Mapping. a. Run Nginx container. b. Map host port 8080 to container port 80. c. Verify application in browser.

	d. Run another Nginx container on a different host port. 2. Docker Networking a. Create a bridge network. b. Launch two Ubuntu containers. c. Connect both to the same network. d. Verify communication using ping. 3. Data Persistence Using Volumes. a. Create a volume. b. Mount it to a container. c. Create files inside mounted directory. d. Remove container and verify data persistence. 4. Bind Mounts for Host-Container Sharing (B) a. Create a website page on host. b. Mount it inside an Nginx container. c. Verify changes through browser.
LAB-6	Docker 1. Building Custom Docker Images. a. Create a simple HTML page. b. Write Dockerfile. c. Build image. d. Run container. 2. Multi-Container Application using Docker Compose a. Create Docker Compose file. b. Start MySQL and phpMyAdmin containers. c. Verify communication between services.
Lab-7	Docker 1. Containerize and run a full-stack web application using Docker Compose by integrating frontend, backend, and database services.
LAB-8	Postman 1. Exploring API Fundamentals using Postman a. Install and configure Postman. b. Send GET requests to a public API such as: JSONPlaceholder, ReqRes c. Observe: Status Code, Headers, Response Body d. Compare responses of: GET, POST, PUT, DELETE 2. Creating and Managing Requests & Collections. a. Create a collection named Student Management API. b. Add requests: Get all students, Get student by ID, Add student c. Create folders inside the collection. d. Save all requests into appropriate folders. e. Export and import the collection.

LAB-9	Postman - Working with Query Parameters, Path Variables and Request Bodies - Send GET requests using: Query parameters, Path parameters - Create POST requests using: JSON body, Form-data - Modify and resend requests with different values. - Dynamic Workflows using Environment Variables - Create variables: base_url, user_id, token - Replace hardcoded values with variables. - Switch between environments and verify results. - Writing Postman Scripts and Automated Tests (B) - Add test scripts to verify: - Status code = 200 - Response time < 1000 ms - Required fields exist - Write JavaScript assertions using pm.test(). - Display values in Postman Console
LAB-10	Postman Test a Mock API using Postman and execute the collection to verify API responses.
LAB-11	Jira - Creating and Managing a Jira Project - Login to Jira. Create a new Team-Managed Scrum project named College ERP Project. - Create project summary and key. - Explore: Backlog, Board, Timeline, Reports, Project Settings - Agile Backlog Management - Create: 3 Epics 5 User Stories 5 Tasks 3 Bugs - Link Stories with Epics. - Assign priorities. - Estimate Story Points. - Arrange backlog according to priority.
LAB-12	Agile - Working as an Agile Team - Create a team of 3–4 users. - Assign issues to team members. - Add comments on issues. - Mention team members using @mentions. - Upload attachments. - Watch and vote on issues.

	2. Searching and Using Filters - Search issues assigned to yourself. - Search all high-priority issues. - Search completed issues. - Create and save filters: My Open Issues, High Priority Bugs - Share filters with teammates.
LAB-13	Figma - Introduction to Figma Interface and Basic UI Design - Create a new Figma file. - Design a Login Screen containing: - Logo - Username field - Password field - Login button - Apply appropriate alignment and spacing. - Export the design as PNG. - UI/UX Principles Analysis - Select any existing website/app screen. - Identify: - Color issues - Alignment issues - Typography inconsistencies - Redesign the screen in Figma following UI/UX principles. - Layout Design using Frames and Auto Layout - Design a Dashboard page containing: - Header - Sidebar - Main content area - Footer - Apply Auto Layout to all sections. - Ensure proper spacing and padding.
LAB-14	Figma - Responsive Mobile and Tablet Design - Create a Mobile App Home Screen. - Duplicate it for Tablet view. - Adjust: - Layout - Image sizes - Text sizes - Use Constraints and Auto Layout. - Components and Reusable Elements - Design:

	- Primary Button - Secondary Button - Text Field - Navigation Bar b. Convert all elements into Components. c. Create at least 5 instances of each component. 3. Styling and Visual Consistency a. Design three screens: - Login - Dashboard - Profile b. Use shared: - Colors - Fonts - Buttons - Icons c. Create Style Libraries.
LAB-15	Figma 1. E-Commerce App UI Design a. Design the following screens: - Home Page - Product Listing - Product Details - Shopping Cart b. Requirements: - Consistent styling - Auto Layout - Reusable components 2. Interactive Prototyping a. Design: - Splash Screen - Login Screen - Home Screen - b. c. Connect screens using Prototype mode. d. Add: - Navigate To interactions - Smart Animate transitions - Button click actions